Vulnerabilità

Description

Un difetto (bug, errore, ...) di tipo:

- hardware
- software
- nelle persone
- nelle regole della politica

che permette di violare la politica di sicurezza = un soggetto riesce ad eseguire una operazione per cui non ha diritti

> 💡 Tutte le vulnerabilità sono difetti, non tutti i difetti sono vulnerabilità

Ciclo di vita di una vulnerabilità

1. Nasce quando qualcuno commette un errore
2. Diventa nota quando viene scoperta da qualcuno
3. Diventa pubblica quando la sua presenza viene rivelata

A questo punto iniziano due attività parallele

- Ricerca di un modo per eliminarla
- Ricerca di un exploit che la sfrutti

In base a chi arriva prima può diventare una vulnerabilità sfruttata. Quando è disponibile un modo per eliminarla deve essere applicato.

💡 Negli USA sono stati creati database con vulnerabilità e con vulnerabilità sfruttate

Il modo più diffuso per eliminare una vulnerabilità è quello di «patcharla»

Attacco

Azioni + Esecuzione di un programma che permette ad una minacciadi ottenere diritti vietati dalla politica di sicurezza.

Esempio:
programma riceve in input un valore e lo usa per accedere ad una posizione di memoria ma non controlla il valore ricevuto in ingresso se trasmetto indirizzo di una informazione critica posso leggerla

💡 Un attacco ha successo perché esiste una vulnerabilità

Exploit: Programma eseguito in un attacco

Automatizzabile: Un attacco che richiede solamente esecuzione di codice (sono i più critici perché possono essere eseguiti da tutti)

Intrusione in un sistema informatico

In una intrusione informatica una minaccia esegue una sequenza di azioni/attacchi per raggiungere obiettivo = controllo di oggetti del SI

Intrusione automatizzata = le azioni dell'intrusione possono esserecodificate in o guidate da un programma e tutti attacchi sonoautomatizzabili

Quando l'attaccante raggiunge obiettivo subentra al proprietario (owner) del sistema e allora può:

- **C** - raccogliere informazioni
- **I** - modificare informazioni

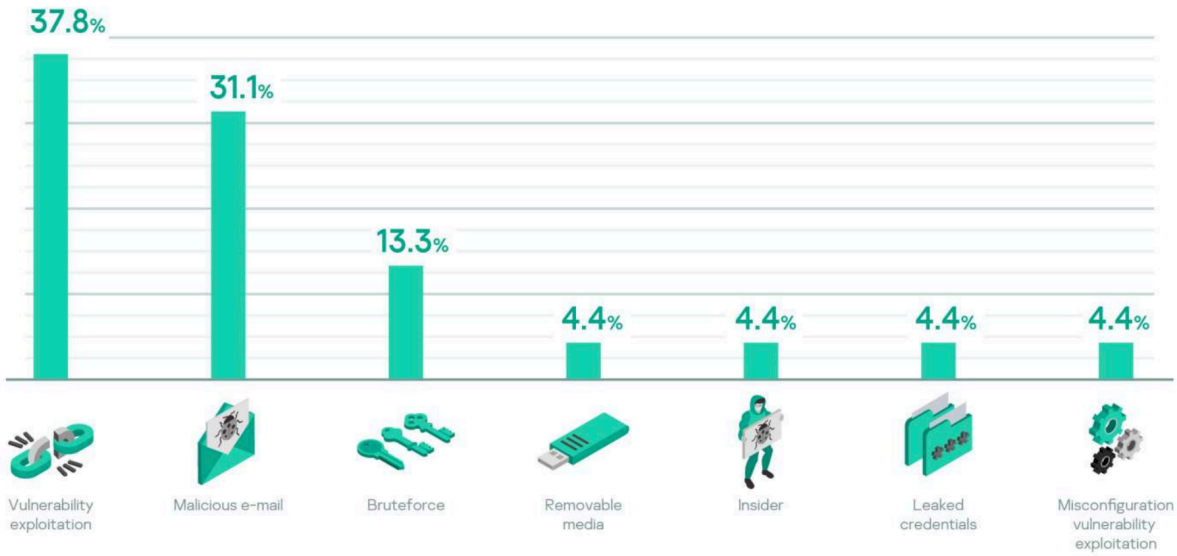
- **A** - impedire ad altri di accedere alle informazioni

Fasi di intrusione

1. Attaccante raccoglie informazioni iniziali sul sistema
2. Individuazione (scoperta) delle vulnerabilità del sistema per accesso iniziale3. Intrusione = sequenza di azioni/attacchi repeat
3. raccolta informazioni su sistema
4. Scoperta vulnerabilità nei component sistemi
5. Costruzione Exploit
6. Azione/Attacco: Esecuzione dell'exploit + Eventuali azioni umane(attaccante può ripeterlo se non ha successo) until raggiunto obiettivo
7. Installazione di strumenti per il controllo (persistenza)
8. Cancellazione delle tracce dell'intrusione
9. Accesso, modifica, ..., ad un sottoinsieme delle informazioni del sistema o altri attacchi
    a. Information stealing = furto = leaking=exfiltration
    b. Cifra e chiedi riscatto

Questa fase può essere vista come una escalation di privilegi (attacchi) ed informazioni fino ad ottenere tutte le informazioni che permettono di raggiungere obiettivo ed i diritti che permettono di controllarlo

Indicata in generale come privilege escalation = attacchi aumentano i diritti o privilegi ma non si deve trascurare la raccolta di informazioni che pilota la scelta di quali privilegi ottenere e quindi quali attacchi eseguire



Approcci alla sicurezza

Per colpa delle vulnerabilità esistono le intrusioni e ciò ci porta quindi a definire due approcci alla sicurezza:

Sicurezza incondizionale

1. Si assume che qualsiasi sia la vulnerabilità nel sistema esista qualcuno interessato ad usarla ed in grado di farlo
2. Quindi tutte le vulnerabilità vanno eliminate

Sicurezza condizionale (*analisi e gestione del rischio*)

1. Richiede analisi delle minacce (chi è interessato ad attaccare il mio sistema e sa come attaccarlo)
2. Eliminare solo quelle vulnerabilità che una minaccia reale può utilizzare per attaccare il sistema (in una intrusione)

Analisi del rischio

Alternativo alla sicurezza incondizionale perchè esso spesso provoca aumenti non accettabili del costo di un sistema informatico, è un approccio guidato all'economia per capire ***cosa possiamo difendere*** e cosa ***conviene difendere***

1. Analisi delle risorse da proteggere (asset)
2. Analisi degli minacce = sicurezza condizionale
3. Analisi delle vulnerabilità
4. Analisi degli attacchi
5. Analisi degli impatti
6. Individuazione rischio, rischio accettabile ed introduzione delle contromisure (gestione del rischio)

Analisi delle risorse da proteggere

1. Individuare un insieme di oggetti(= risorse fisiche, logiche, immateriali di interesse per l'azienda/ente) ed alcune proprietà di sicurezza
2. Definire una politica di sicurezza su questi oggetti in termini delle proprietà di sicurezza precedenti:
    a. Chi può leggere alcune informazioni
    b. Chi può modificare le informazioni
    c. Chi può eseguire certi programmi

Queste proprietà diventano gli obiettivi di sicurezza che il sistema deve garantire

Analisi delle minacce

Individuare chi è interessato ad attaccare il sistema che è target dell'analisi per

- Rubare informazioni (proprietà intellettuale e piani)
- Modificare informazioni (controllo industriale)
- Impedire agli utenti di utilizzare sistemi (e-commerce, ransomware)

Possibili minacce

- Intelligenti (con un piano, delle abilità e degli obiettivi)
- Competitor

- Criminalità organizzata (CaaS)
- Stati (APT)
- Terroristi
- Clienti
- Eventi naturali (safety)

Safaty vs Security

💡 **Safety**: capacità di resistere ad eventi di origine non umana e casuale (inondazioni, terremoti, crolli) e non esiste malizia nell'evento

💡 **Security**: capacità di resistere ad attacchi di origine intelligente che fanno parte di un piano malizioso per raggiungere un obiettivo predefinito da chi esegue gli attacchi

Analisi delle vulnerabilità / attacchi

Quali sono le vulnerabilità nei componenti del sistema che permettono ad un attaccante (minaccia) di ottenere, in un certo numero di passi, accesso alle risorse di suo interesse.

Le vulnerabilità possono essere scoperte:

- in modo casuale o manuale
- da locale o da remoto

Un attacco è un azione che ha una certa probabilità di successo, se ha successo aumenta i diritti ovvero i privilegi dell'attaccante

💡 E’ fondamentale per i difensori scoprire le vulnerabilità prima degli attaccanti per poter rimediare alcune vulnerabilità perché in generale non è necessario rimediare tutte le vulnerabilità

Attributi di un attacco:

- Diritti richiesti/diritti ottenuti
- Probabilità di successo
- Competenze/Abilità richieste
- Rumore generato = Modifiche di alcune proprietà del sistema = Riduzione delle prestazioni, aggiornamento di file o altre informazioni
- Automatizzato/Automatizzabile/Non automatizzabile
- Locale/Remoto

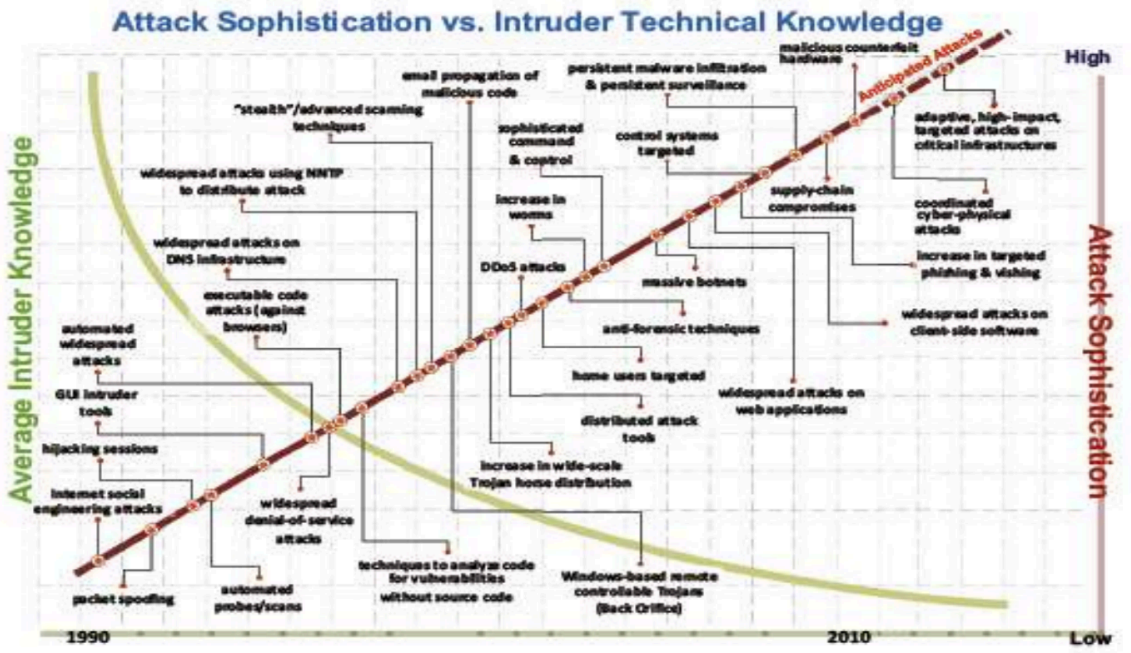
Alcuni approcci associano una valutazione di pericolosità ad un attacco e quindi alla vulnerabilità che lo abilita

Questa valutazione non è consistente con idea di intrusione perché come abbiamo visto intrusione richiede una sequenza di azioni ed attacchi e quindi non è possibile una valutazione indipendente dal contesto del sistema in cui la vulnerabilità appare

Attacco / azione automatizzabile

Attacco non automatizzabile è strutturalmente meno pericoloso di uno automatizzabile, in questo caso le azioni umane non sono necessarie, bastaeseguire un programma.

Una intrusione è automatizzabile se è automatizzabile ogni azione che utilizza e la scelta delle azioni (ci si aspetta un contributo da AI)



Attacco locale

Se si dispone di un account sul sottosistema, ovvero essere utenti del sistema o aver rubato le credenziali

Attacco remoto

Non si dispone di un account sul sottosistema

> 💡 Un attacco remoto e automatizzabile è il più pericoloso perché può essere realizzato da chiunque e lanciato da qualsiasi elemento in rete

Buffer Overflow

La vulnerabilità sicuramente più comune tra quelle del C ed è praticamente assente in linguaggi di alto livello che non permettono al programmatore la gestione della memoria.

Trasmette ad una funzione di un certo modulo software una quantità di informazioni superiore a quella prevista dal programmatore.

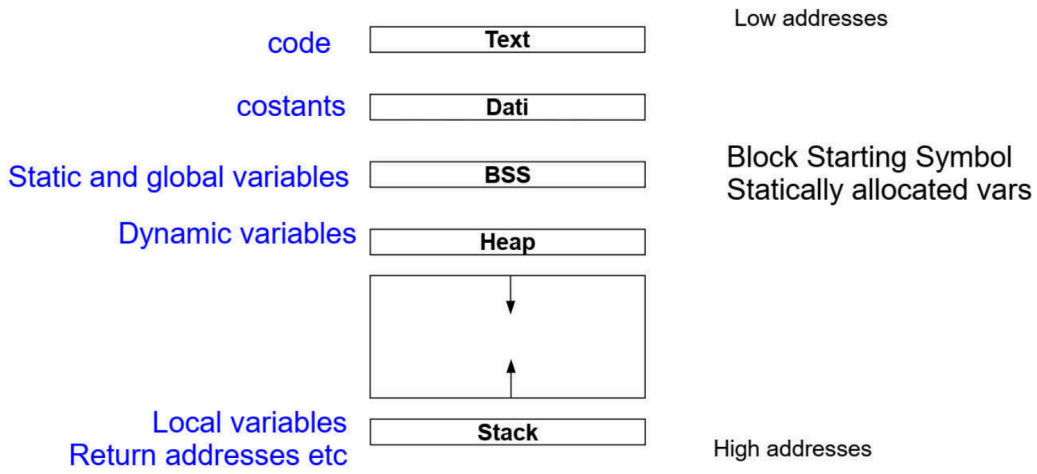
Se il software è privo di controlli è possibile trasmettere del codice eseguibile (code injection) ed eseguirlo, ciò può garantire il pieno possesso di tutte le funzionalità del sistema.

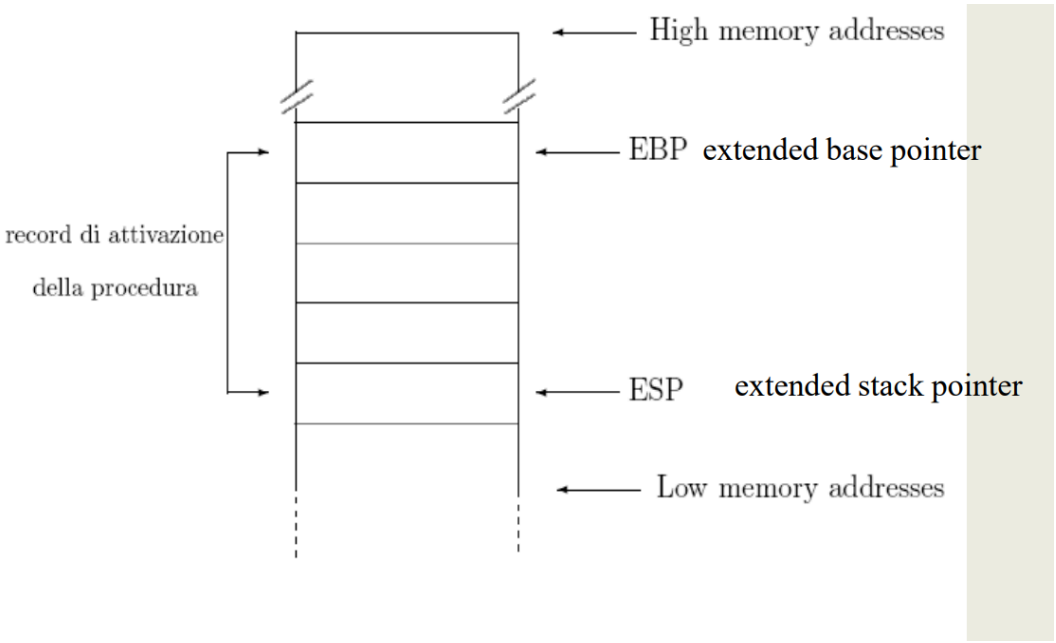
Memoria di un processo

Il *text segment* ha dimensione fissa, memorizza il codice del programma ed è in sola lettura. Ogni tentative di scrittura provoca un segmentation error (segmentation fault – core dumped)

Il *data segment* memorizza variabili statiche e dinamiche

Lo *stack segment* memorizza i dati per gestire call e return di funzioni





Analizziamo quest'esempio di codice in C per capire meglio l'allocazione dello *stack frame*:

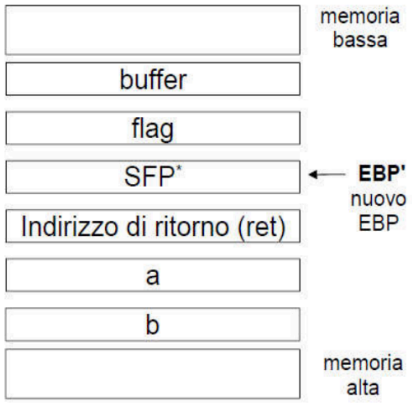
```
void test_function (int a, int b)
{
  char flag;
  char buffer[10];
}

int main()
{
  test_function (1,2);
  exit(0);
}
```

EIP

Stato dello stack frame

Indirizzo di ritorno (ret) = EIP + 4 byte



*SFP= Saved Frame Pointer, valore utilizzato per rispristinare lo stato originale di EBP (prima della chiamata di test_function();)

Esempio di Overflow:

```
void overflow_function (char *str) {
  char buffer[20];

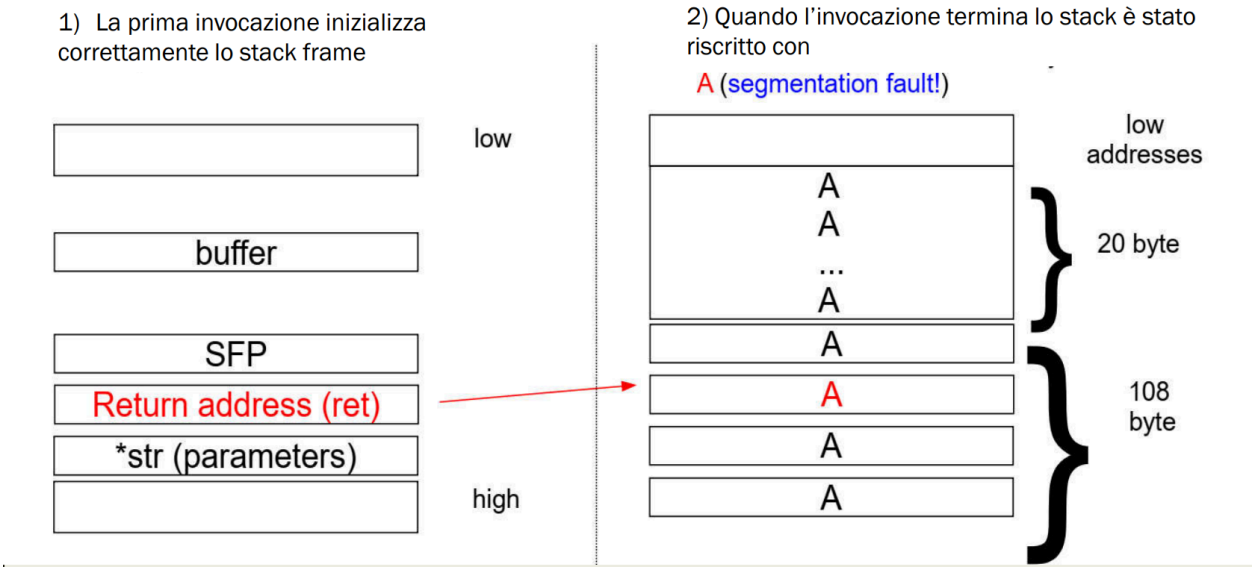
  strcpy(buffer, str);    Questa funzione copia str in buffer
}

int main() {
  char big_string[128];
  int i;

  for(i=0; i < 128; i++)
  {
    big_string[i] = 'A';
  }
  overflow_function(big_string);
  exit(0);
}
```

Questo produce un overflow

Il precedente codice può invocare un segmentation fault:



Attacco stack based overflow

Sfrutta il ret:

> Il `ret` è un'istruzione in linguaggio assembly che serve a **ritornare** dal contesto di una funzione. Quando viene eseguita, **ripristina l'indirizzo di ritorno** (che si trova nello stack) e trasferisce l'esecuzione del programma a

> quel punto, quindi il controllo torna alla funzione chiamante. In pratica, termina l'esecuzione della funzione corrente e restituisce il flusso di esecuzione al punto in cui era stata chiamata.

Sostituendo ret con un puntatore a del codice iniettato (scritto) all'attaccante in una zona di memoria, spesso nello stack stesso, questo attacco assume la capacità di manipolare l'indirizzo di ritorno e di iniettare del codice nella memoria del sistema

Buffer overrun/overflow

Il **buffer overflow** (o **buffer overrun**) è un errore che si verifica quando un programma scrive più dati di quelli che può contenere un buffer (un'area di memoria allocata per immagazzinare dati).

Questo provoca la sovrascrittura di dati adiacenti, che può corrompere la memoria, causare crash del programma o creare vulnerabilità di sicurezza, come l'esecuzione di codice arbitrario.

Un esempio comune è quando si inserisce una stringa troppo lunga in un array di dimensioni fisse.

Vulnerabilità dello stack overflow

1. Manca il controllo sulla dimensioni delle variabili
2. Cattivo sistema di tipi
3. Operazione di memoria errata
4. Errata dimensione dello stack

Vulnerabilità strutturali `TCP/IP`

Esempio:

1. **R** è una rete di 1000 nodi, con indirizzo parziale **X**, un indirizzo parziale comune

a tutti i nodi di R

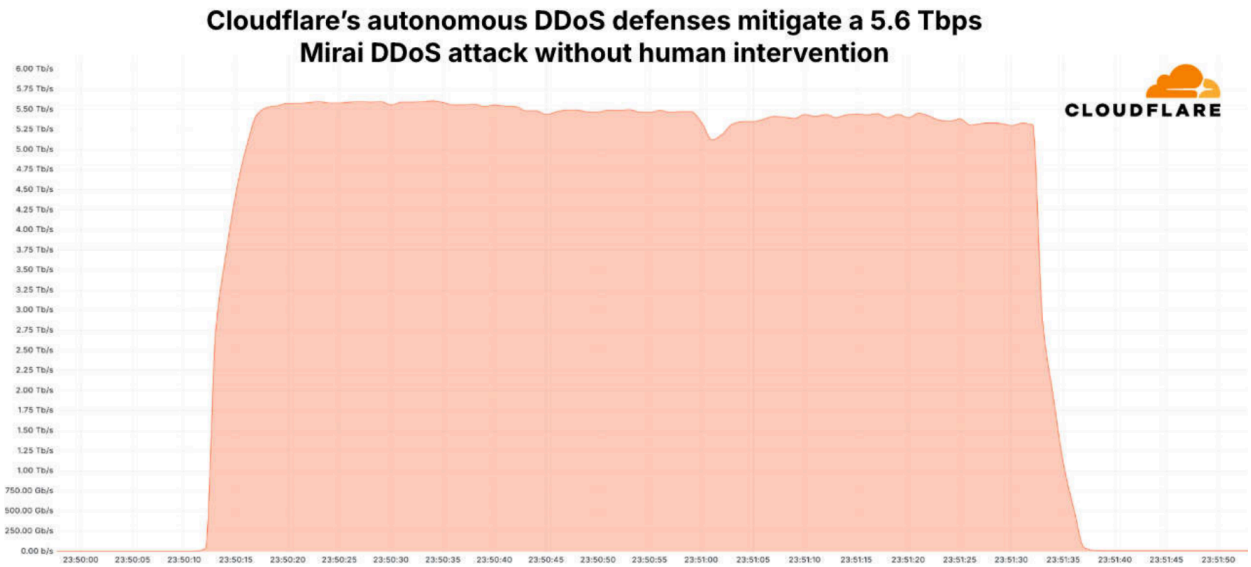
2. Il nodo **A** manda un messaggio di `ECHO` ad indirizzo **X** dicendo di essere il nodo **B**
3. Tutti i nodi di **R** rispondono a **B**
4. **B** per un certo tempo non può comunicare perché le sue linee di comunicazione sono intasate

Tramite questi passaggi si ottiene un attacco ***DDoS***
Un **Distributed Denial of Service (DDoS)** è un attacco informatico in cui un gran numero di dispositivi compromessi (spesso in una botnet) viene utilizzato per inviare traffico massivo a un server o una rete, sovraccaricandola e rendendola inaccessibile agli utenti legittimi.

L'obiettivo è **interrompere il servizio** e causare disservizi.

L'effetto si può amplificare a piacere aumentando il numero di nodi che hanno il ruolo di A (zombies)

Questo è un tipico attacco da cui è estremamente difficile difendersi perché B non ha un modo di scoprire che l'attacco è in preparazione



Contromisure per attacchi DDoS

Le **contromisure** contro un attacco **DDoS** includono:

1. **Filtraggio del traffico**: Utilizzare firewall e sistemi di filtraggio avanzati per identificare e bloccare il traffico sospetto.
2. **Distribuzione del traffico**: Utilizzare **Content Delivery Networks (CDN)** e **load balancers** per distribuire il traffico su più server, riducendo il rischio di sovraccarico su uno solo.
3. **Rilevamento e mitigazione automatica**: Implementare sistemi di monitoraggio che rilevano attacchi in tempo reale e attivano misure di mitigazione automatiche.
4. **Cloud-based DDoS protection**: Affidarsi a servizi di protezione DDoS offerti da provider cloud che hanno capacità scalabili per gestire picchi di traffico.
5. **Limiti di connessione**: Imporre limiti sul numero di connessioni simultanee per ridurre l'impatto del traffico dannoso.

L'idea è ridurre la capacità di attaccare il sistema e mantenerlo operante nonostante l'attacco.

Classificazione delle vulnerabilità

- **Procedurale**: azioni mal eseguite
  - Una password comunicata per iscritto in una busta non sigillata
- **Organizzative**: azioni ben definite ma eseguite male
  - Amministratori diversi e contemporanei per la stessa macchina
  - Compiti assegnati a persone non formate
- **Strumenti**: hardware o software
  - Una password trasmessa in chiaro per email

Classificazione delle vulnerabilità negli strumenti

Una diversa classificazione, utile anche se spesso non rigorosa e potenzialmente ambigua

- **Specifica**: Uno strumento che è più generale di quanto richiesto
  - Uso di una libreria che comprende più funzioni del necessario (*se qualcuno riesce ad invocare una funzione di quelle inutili si possono avere problem di sicurezza*)
  - Il riuso di codice può portare al riuso delle vulnerabilità.

    💡 **Hardening del codice**
    rimozione dal codice di tutte quelle parti che sono inutili o introdotte solo per debugging

- **Implementazione**: Un errore nel codice di uno strumento
  - nessun controllo su Input utente, parametri di funzioni/metodi
- **Strutturale**: Il comportamento anomalo o vulnerabile nasce quando alcuni strumenti vengono integrati in un unico sistema
  - Alcuni moduli delegano ad altri i controlli di sicurezza.

Ciclo di vita di una vulnerabilità

1. Nasce con un errore
2. Diventa nota quando viene scoperta da qualcuno
3. Diventa pubblica quando la sua presenza viene rivelata
4. A questo punto iniziano due attività parallele
   a. Ricerca di un modo per eliminarla
   b. Ricerca di un exploit che la sfrutti

💡 Il modo più diffuso per eliminare una vulnerabilità è quello di **patcharla**

Patch/Patching

Le patch per il software proprietario solitamente sono file eseguibili e non codice sorgente.
La loro esecuzione apporta le modifiche ai file binari o sostituisce gli stessi file binari.

Le patch possono anche essere rilasciate sotto forma di file di testo (`diffs.txt`) contenenti le differenze, le quali poi vengono elaborate dai gestori di versione (`CVS`), che generano in automatico la patch.
In questo caso l'utente deve compilare i nuovi sorgenti. Soluzione solitamente adottata nel caso di software open source

Una nuova patch può però introdurre problematiche che richiedono una regressione della versione software dell'applicativo

Tipologie di vulnerabilità

- **Vulnerabilità note**: vulnerabilità pubbliche in moduli noti
- **Vulnerabilità non pubbliche**: vulnerabilità scoperte ma non pubblicate, possono ad esempio apparire in database a pagamento
- **Zero day**: vulnerabilità scoperte e non pubblicate ed usate da organizzazioni e stati negli attacchi delle loro intrusioni

Vulnerability Scanner

Uno strumento informatico che permette di ***costruire l'inventario*** e conoscere le vulnerabilità note dei moduli dei vari nodi

Lo strumento opera su un certo sottoinsieme di nodi e restituisce:

- I moduli di ogni nodo che interagiscono con gli altri moduli
- Le vulnerabilità di questi moduli

Invia pacchetti sulla porta e dall’analisi delle risposte deduce quale modulo sta operando su quella porta

Confusione Matrix

Matrice nata in medicina per analizzare fenomeni complessi che non possono essere risolti utilizzando un metodo booleano

